

Learning API Calls: The 20% That Matters

1 The Mental Model

An API call is just sending an HTTP request to a URL and reading the response. Every API interaction has four parts:

- **Method** — what you want to do (GET, POST, etc).
- **URL** — where to send it, possibly with query parameters.
- **Headers** — metadata (auth tokens, content type).
- **Body** — the data you send (usually JSON, only for POST/PUT/PATCH).

The server replies with a **status code**, **response headers**, and a **body** (usually JSON).

2 HTTP Methods (CRUD Mapping)

- GET — read data. No body. Idempotent.
- POST — create new resource. Has body.
- PUT — replace existing resource. Has body.
- PATCH — partially update a resource. Has body.
- DELETE — remove a resource.

3 Status Codes (Recognize These Instantly)

- 2xx success — 200 OK, 201 Created, 204 No Content.
- 3xx redirect — usually handled automatically.
- 4xx your fault — 400 Bad Request, 401 Unauthorized (no/invalid auth), 403 Forbidden (auth ok, not allowed), 404 Not Found, 429 Too Many Requests (rate limit).
- 5xx server fault — 500 Internal Error, 502 Bad Gateway, 503 Unavailable.

4 The Tool: Python requests

This is the default library. Install once: `pip install requests`.

```
1 import requests
2
3 # GET
4 r = requests.get("https://api.github.com/users/torvalds")
5
6 # Inspect the response
7 print(r.status_code)    # 200
8 print(r.headers)        # response headers (dict-like)
9 print(r.text)           # raw body as string
10 print(r.json())         # body parsed as Python dict (if JSON)
```

5 Query Parameters (for GET)

Pass a dict to `params=` — requests builds the URL for you.

```
1 r = requests.get(
2     "https://api.github.com/search/repositories",
3     params={"q": "pytorch", "sort": "stars", "per_page": 5}
4 )
5 # Final URL: .../search/repositories?q=pytorch&sort=stars&per_page=5
6
7 data = r.json()
8 for repo in data["items"]:
9     print(repo["full_name"], repo["stargazers_count"])
```

6 Headers and Authentication

Most real APIs require authentication. Almost always one of two patterns:

```
1 # 1. Bearer token (most modern APIs: OpenAI, Anthropic, GitHub, etc.)
2 headers = {
3     "Authorization": "Bearer YOUR_TOKEN_HERE",
4     "Content-Type": "application/json"
5 }
6 r = requests.get("https://api.example.com/me", headers=headers)
7
8 # 2. API key in a custom header
9 headers = {"X-API-Key": "YOUR_KEY_HERE"}
10 r = requests.get("https://api.example.com/data", headers=headers)
```

Never hardcode keys. Use environment variables:

```
1 import os
2 token = os.environ["API_TOKEN"]
3 headers = {"Authorization": f"Bearer {token}"}
```

7 POST with a JSON Body

```
1 payload = {"title": "Hello", "body": "World", "userId": 1}
2
3 r = requests.post(
4     "https://jsonplaceholder.typicode.com/posts",
5     json=payload,          # <-- use json=, NOT data=, for JSON
6     headers={"Authorization": "Bearer ..."}
7 )
8
9 print(r.status_code)      # 201
10 print(r.json())          # the created object echoed back
```

Key detail: `json=payload` automatically serializes the dict to JSON *and* sets `Content-Type: application/json`. Using `data=payload` sends it as a form, which is different.

8 The Full CRUD Set

```
1 requests.get(url, params=...)
2 requests.post(url, json=...)
3 requests.put(url, json=...)
4 requests.patch(url, json=...)
5 requests.delete(url)
```

9 Error Handling (Do This Always)

```
1 try:
2     r = requests.get(url, timeout=10)    # ALWAYS set a timeout
3     r.raise_for_status()                 # raises if 4xx/5xx
4     data = r.json()
5 except requests.Timeout:
6     print("Request timed out")
7 except requests.HTTPError as e:
8     print(f"HTTP error: {e.response.status_code}")
9 except requests.RequestException as e:
10    print(f"Network error: {e}")
```

The two non-negotiables: a `timeout` and a `raise_for_status()` check.

10 Sessions (Faster + Cleaner)

For multiple calls to the same API, use a `Session`. It reuses the TCP connection and lets you set common headers once.

```
1 s = requests.Session()
2 s.headers.update({"Authorization": f"Bearer {token}"})
3
4 r1 = s.get("https://api.example.com/users")
5 r2 = s.get("https://api.example.com/posts")
```

11 Reading curl in Docs

API docs often show `curl` commands. Translate them mentally:

```
1 curl -X POST https://api.example.com/items \
2   -H "Authorization: Bearer TOKEN" \
3   -H "Content-Type: application/json" \
4   -d '{"name": "thing", "qty": 3}'
```

becomes:

```
1 requests.post(
2     "https://api.example.com/items",
3     headers={"Authorization": "Bearer TOKEN"},
4     json={"name": "thing", "qty": 3}
5 )
```

Mapping: `-X` = method, `-H` = headers, `-d` = body.

12 Rate Limiting and Retries

APIs limit how often you can call them. If you hit 429, check the `Retry-After` header.

```
1 import time
2
3 for attempt in range(5):
4     r = requests.get(url, headers=headers)
5     if r.status_code == 429:
6         wait = int(r.headers.get("Retry-After", 2 ** attempt))
7         time.sleep(wait)
8         continue
9     r.raise_for_status()
10    break
```

13 Pagination (Common Pattern)

APIs return results in pages. Two typical styles:

```
1 # Page-number style
2 page = 1
3 while True:
4     r = requests.get(url, params={"page": page, "per_page": 100})
5     items = r.json()
6     if not items:
7         break
8     process(items)
9     page += 1
10
11 # Cursor/next-URL style (look for "next" in response)
12 next_url = url
13 while next_url:
14     r = requests.get(next_url, headers=headers)
15     data = r.json()
16     process(data["results"])
```

```
17 next_url = data.get("next")    # None when done
```

14 Async Calls (When You Need Speed)

For many parallel requests, switch to `httpx` (same API style as `requests`).

```
1 import asyncio, httpx
2
3 async def fetch(client, url):
4     r = await client.get(url)
5     return r.json()
6
7 async def main():
8     urls = [f"https://api.example.com/items/{i}" for i in range(100)]
9     async with httpx.AsyncClient() as client:
10         results = await asyncio.gather(*(fetch(client, u) for u in urls))
11
12 asyncio.run(main())
```

15 Real Example: Calling an LLM API

The pattern is identical regardless of provider.

```
1 import os, requests
2
3 r = requests.post(
4     "https://api.openai.com/v1/chat/completions",
5     headers={
6         "Authorization": f"Bearer {os.environ['OPENAI_API_KEY']}",
7         "Content-Type": "application/json"
8     },
9     json={
10         "model": "gpt-4o-mini",
11         "messages": [{"role": "user", "content": "Hello!"}]
12     },
13     timeout=30
14 )
15 r.raise_for_status()
16 print(r.json()["choices"][0]["message"]["content"])
```

16 The Checklist for Any New API

1. Find the **base URL** and the **endpoint** for your task.
2. Identify the **auth method** (Bearer token? API key header?).
3. Find the **method** (GET / POST / ...).
4. Note required **params** (query) or **body fields** (JSON).
5. Check the **response shape** so you know what keys to read.
6. Note the **rate limits**.

17 Practice Task

Use the free public API <https://jsonplaceholder.typicode.com>:

1. GET `/posts/1` and print its title.
2. GET `/posts` with `params={"userId": 1}` and print how many came back.
3. POST a new object to `/posts` and print the returned id.
4. Wrap everything in `try/except` with a timeout and `raise_for_status()`.